

20. Binary Search :

Given Dataset :-

[3, 8, 14, 19, 25, 31, 40]

Target Element : 3

187.

1. Problem Understanding :-

The problem is to locate the element 3 in sorted dataset using Binary Search algorithm. Since the data is arranged in ascending order. Binary Search can efficiently find the target by repeatedly dividing the search interval into two halves instead of checking every element one by one.

2. Method Selection :-

- dataset is sorted ascending order and reduces the search space by half after every comparison.
- The algorithm has a time complexity of  $O(\log n)$
- So binary search is selected.

Step by step solution :-

[3, 8, 14, 19, 25, 31, 40]

Target = 3

① Low = 0

high = 6

$$\text{Mid} = \frac{(0+6)}{2} = 3$$

Mid element = 19

$3 < 19 \Rightarrow$  search the left half

② Low = 0

high = 2

$$\text{Mid} = \frac{(0+2)}{2} = 1$$

Mid element = 8

$3 < 8 \Rightarrow$  search the left half

③ Low = 0

high = 0

$$\text{Mid} = (0+0)/2 = 0$$

Mid element = 3

$3 = 3$

$\therefore$  Element found

Calculations :-

| step | Low | high | Mid | Mid element | Decision      |
|------|-----|------|-----|-------------|---------------|
| 1    | 0   | 6    | 3   | 19          | search left   |
| 2    | 0   | 2    | 1   | 8           | search left   |
| 3    | 0   | 0    | 0   | 3           | Element found |

Total Comparisons = 3

Target Element: 3

Position (Index): 0

Result: Element found

Interpretation of Result:-

- Binary search found the target element 3 in 3 Comparisons by repeatedly dividing the search interval.
- Although the element is at beginning, the algorithm checks the middle first.
- For large sorted datasets, Binary search is more efficient with  $O(\log n)$  time Complexity compared to  $O(n)$  for linear search.

Time Complexity:-

- Best Case:  $O(1)$
- Average Case:  $O(\log n)$
- Worst Case:  $O(\log n)$